HTTP/3 Analysis and Attack: Spoofed-ACK via QUIC

Elad Imany
Department Of Computer Science
Ariel University
Ariel, Israel
elad.imany@msmail.ariel.ac.il

Orel Shalem
Department Of Computer Science
Ariel University
Ariel, Israel
orel.sharabi1@msmail.ariel.ac.il

***Abstract*—The modern Internet relies on advanced communication protocols that enable fast, reliable, and secure data transfer between users, browsers, and servers. Over the past decades, the HTTP protocol has undergone significant upgrades—from HTTP/1.1, through HTTP/2, to the most recent version, HTTP/3. This latest protocol is built from the ground up on top of QUIC, a UDP-based transport protocol developed by Google to enhance browsing performance, reduce latency, and ensure stable operation even over unreliable networks or in the presence of packet loss.**

Alongside technological advancements, the cybersecurity landscape presents new challenges at every stage. Each new protocol—particularly one replacing an existing foundation such as TCP—provides fertile ground for the emergence of novel attacks. In many cases, performance-oriented features such as faster connection establishment (handshake), reduced initial encryption overhead, or optimized data compression may come at the expense of security, introducing new attack vectors.

This project aims to examine the resilience of HTTP/3 and identify vulnerabilities in QUIC, with a particular focus on the Spoofed ACK attack—a sophisticated Denial of Service (DoS) technique based on crafting forged ACK messages that cause a server to transmit a large volume of retransmitted packets to a spoofed target. This attack leverages an in-depth understanding of QUIC packet structures, mastery of encryption and decryption processes, and precise real-time packet timing.

The research includes a theoretical review, mapping of known attacks, analysis of relevant RFCs, and the construction of a Docker-based experimental environment comprising a server, client, and attacker. Advanced tools such as Wireshark, QLOG, NFQUEUE, and Scapy were utilized, and open-source code from the QUIC Forge paper was adapted to develop the attack scenario.

This report presents the research findings, experimental methodology, and encountered challenges, and concludes with practical insights on strengthening QUIC security and enhancing HTTP/3's resilience against advanced attacks.

***Index Terms*—HTTP/3, QUIC, Cybersecurity, Denial of Service, Spoofed ACK, Network Security**

I. Introduction

A. Motivation and Scope

HTTP/3 is the most advanced protocol in the HTTP family, based on the QUIC protocol, designed to improve efficiency, reliability, and security in network communication. It was developed to address the challenges presented by previous versions of HTTP, such as head-of-line blocking, high latency, and efficient resource management.

In our research, we focus on analyzing the security mechanisms of HTTP/3 and examining its potential vulnerabilities. The project's goal is to identify whether new attacks can be applied to the protocol and to propose defense mechanisms that can address existing attacks that still lack a satisfactory solution. The main challenge lies in understanding the protocol's complex infrastructures, such as header compression mechanisms, independent streams, and the establishment of secure connections, and in identifying potential weaknesses in different implementations.

B. Overview of HTTP/3 and QUIC

[1] HTTP/2 introduced significant innovations, such as multiple streams in a single connection, but relied on the TCP protocol, causing issues like head-of-line blocking. HTTP/3, based on the QUIC protocol, redefines connection management and offers advanced capabilities including faster connections, better handling of unstable networks, and completely independent streams.

QUIC Protocol QUIC, on which HTTP/3 is based, is a modern transport protocol developed by Google and replaces TCP. It is optimized for low latency, built-in security, and independent multiplexing of streams. Key features of QUIC include:

- **Integration with TLS 1.3:** Ensures a secure and reliable connection while reducing the setup time.
- **Handling packet loss:** QUIC allows retransmission of only the lost data without affecting other streams in the connection.
- **Use of UDP:** As a transport base, QUIC is not dependent on the order of arrival of packets and offers advantages in unstable networks.

C. Key Features

- **Independent streams:**
  - Independent streams ensure that damage or packet loss in one stream does not affect others.
  - Eliminates the head-of-line blocking issue common in HTTP/2.
- **Header Compression (QPACK):**

  - QPACK, developed specifically for HTTP/3, replaces HPACK and provides an asynchronous compression mechanism.
  - Adapted for high-latency and packet loss networks, preventing stream blocking.
- **Integrated TLS 1.3:** The built-in integration of TLS 1.3 in the QUIC protocol enables quick and secure connections, reducing the latency required for establishing a connection.
- **Modular Frame System:** HTTP/3 is based on well-defined data frames (HEADERS, DATA, SETTINGS), enabling efficient communication management.

D. Attacks and Risks

- **Cross-Protocol Attacks:** Attackers can exploit visible data in QUIC to forge requests or perform spoofing attacks. For example, an attacker might use visible headers or readable data during a QUIC connection to perform Cross-Protocol Request Forgery (CPRF) attacks. This attack could be used to bypass authentication mechanisms on servers not adapted to such protection. Example: Researchers found that attackers could use visible details from QUIC packets to alter HTTP requests and cause the server to perform unwanted actions. Possible solution: Full encryption of all parts of the QUIC packet and protecting authentication mechanisms via TLS.
- **Traffic Analysis Attacks:** Analyzing traffic patterns, such as packet size or frequency, could allow attackers to infer sensitive information about users' activities. For example, analyzing packet sizes could reveal user browsing patterns. Example: Traffic analysis attacks have been used in the past to infer when users accessed sensitive services or specific websites. Possible solution: Implementing random padding mechanisms for packets to obscure patterns and prevent conclusions from being drawn.
- **Header Compression Attacks (QPACK):** Compression mechanisms can lead to length-based attacks, where an attacker guesses sensitive content based on the size of compressed headers. Such attacks have previously exploited vulnerabilities in compression mechanisms such as HPACK. Example: Researchers have demonstrated how to infer sensitive content from header compression by combining malicious requests with regular data. Solution: Separating the compression of public data from sensitive data and using strong encryption to protect the content.
- **Privacy Concerns:** A single QUIC connection allows attackers to analyze traffic patterns and discover information about user activity. For example, an attacker could track the amount of data transmitted in each request to infer which services or websites the user accessed. Example: Such tracking has been used in the past to identify the activities of journalists or activists in countries with internet restrictions. Possible Solution: Using techniques like Mix Networks or intentional delays to obscure activity patterns.

E. Analysis

- **Advantages of HTTP/3 Performance Improvement:** Fast connection establishment, independent streams, and efficient traffic management significantly improve the user experience. Increased reliability: Thanks to QUIC, the protocol efficiently handles network issues such as packet loss or sudden changes in connection quality. Built-in Security: TLS 1.3 provides strong encryption and authentication from the transport layer.
- **Risks and Challenges Cross-Protocol Attacks:** Exposure of visible information in QUIC packets can facilitate spoofing attacks. Privacy Concerns: A single connection for all communications makes it easier to analyze usage patterns and identify user activities. High Resource Requirements: Advanced compression and control mechanisms demand significantly more resources compared to HTTP/2.

F. Recommendations

- **Improving Security Mechanisms:** Using padding strategies and resource management to reduce traffic analysis risks and prevent denial of service.
- **Analyzing and Protecting Existing Vulnerabilities:** Investigating existing attacks and proposing new defense mechanisms to address them.
- **Optimizing the HTTP/3 Protocol:** Focusing on the development of compression mechanisms and intelligent resource allocation to improve performance in diverse networks. HTTP/3 represents a significant breakthrough in internet communication, combining the advanced capabilities of QUIC to address the limitations of HTTP/2. Thanks to accelerated connections, blocking prevention, and improved security, the protocol meets the needs of modern networks. However, it is essential to continue addressing potential risks and adapt innovative security solutions to different implementations of the protocol.

G. Related Work

In this section, we will delve into the body of existing literature and research that forms the foundation of our study. We aim to provide a comprehensive review of previous works in the field that are closely related to our research focus and project objectives. By examining past studies, we can identify established methodologies, uncover critical insights, and recognize the gaps that our work seeks to address. This review not only contextualizes our research within the broader academic landscape but also underscores the significance and novelty of our contributions to the ongoing discourse in this domain.

1) What We Know About HTTP/3 and Its Implementation [2]: The article reviews the HTTP/3 protocol and its integration with the QUIC protocol, comparing it to HTTP/1.1 and HTTP/2. It highlights the advantages of HTTP/3, including

better performance in high-latency networks, elimination of Head-of-Line Blocking, and significant security improvements through the built-in integration of TLS 1.3. Additionally, the article describes the opportunities and challenges in implementing HTTP/3 in cloud environments, IoT, and real-time video networks.

Despite the advantages, the article points out gaps in the research and the broader adoption of the protocol. It notes the vulnerabilities to DDoS attacks and problematic management of UDP traffic, as well as the challenges in detecting and resolving issues using tools like Qlog and Qvis. The research emphasizes that while HTTP/3 represents a significant advancement, there is still room for further research and development to address these challenges and ensure wide and secure usage of the protocol.

2) Security Vulnerabilities and Network Service Disruptions with HTTP/3 [3], [4]: The article analyzes security vulnerabilities and service issues in the HTTP/3 protocol and the QUIC transport protocol on which it is based. The research focuses on the effects of QUIC's migration feature on network devices such as NAT, load balancers (LB), and intrusion prevention systems (IDS/IPS). Additionally, the study highlights the use of the spin bit as a covert channel for transmitting confidential information and presents solutions to mitigate these risks.

a) Research Objectives:

- To investigate how QUIC's connection migration feature affects the performance of critical network devices.
- To demonstrate denial-of-service (DoS) attacks by exploiting connection tracking tables in network devices.
- To analyze the risks of covert information channels implemented through the QUIC spin bit.
- To propose mechanisms to prevent these vulnerabilities.

b) Vulnerabilities:

- **Denial-of-Service (DoS):**
  - Multiple connections through QUIC's connection migration create a load on NAT tables and load balancers.
  - The ability to change connection identifiers makes it difficult to track connections.
- **Covert Channel through Spin Bit:**
  - Allows the transmission of confidential information without being detected by network devices such as firewalls.
  - Used to transfer sensitive data like encryption keys.

c) Attack Types:

- **DoS Attack via NAT:** Exploitation of address translation tables in NAT devices by creating new connections at a high speed.
- **DoS on Load Balancers (LB):** The migration feature creates multiple connections that overload the tables.
- **Covert Channel via Spin Bit:** A channel that allows the transfer of confidential information between endpoints.

d) Experimental Methods:

- **Experimental Setup:** HTTP/3 client and server with aioquic on Windows and Linux systems. A private NAT device simulates an attack scenario.
- **Attack:** Creating new QUIC connections every millisecond to exhaust NAT resources. Similar experiments were conducted on load balancers and IDS/IPS systems.

e) Key Findings:

- DoS attacks via NAT and load balancers led to a complete system resource collapse within less than 65 seconds in intensive scenarios.
- Load balancers lasted an average of 40-50 seconds before complete functional loss.
- The spin bit channel enabled covert data transmission at rates of between 89 and 255 kbps, especially in high-capacity networks (1Gbps).
- IDS/IPS systems struggled to detect attacks exploiting QUIC's migration feature, leading to delays in packet analysis and an inability to prevent attacks in real-time.

f) Mitigation and Protections:

- **NAT Improvements:** Customized NAT tables tracking connection identifiers instead of only IP addresses.
- **Limiting Connection Migrations:** Restricting the number of connection migrations per minute to prevent misuse of the feature.
- **Spin Bit Management:** Detection and reporting of unauthorized use of the spin bit channel through advanced packet monitoring.

g) Conclusions and Implications:

The HTTP/3 protocol provides significant improvements in performance and security, but the research highlights vulnerabilities arising from the migration feature and spin bit. Protocol adjustments and dedicated defense mechanisms are needed to ensure safe implementation in diverse networks. Standards bodies and developers must collaborate to improve the protocol and prevent the exploitation of identified vulnerabilities.

3) A hands-on gaze on HTTP/3 security through the lens of HTTP/2 and a public dataset [5]: HTTP/3, an advancement over HTTP/2, leverages the QUIC protocol to enhance performance, reliability, and security by adopting UDP instead of TCP for communication. Despite improvements, security challenges persist, especially in the context of the vulnerabilities observed in HTTP/2 and their potential applicability to HTTP/3. This paper comprehensively evaluates these challenges.

a) Research Objectives:

- **Analyze HTTP/2 vulnerabilities:** Review known attacks and evaluate their relevance to HTTP/3.
- **Experimentation:** Test HTTP/3-enabled servers against a variety of attacks inspired by HTTP/2 vulnerabilities or newly designed.
- **Dataset Creation:** Compile a labeled dataset of HTTP/2, HTTP/3, and QUIC attacks to facilitate further research.
- **Mitigation Recommendations:** Propose defenses and practical improvements for securing HTTP/3 deployments.

b) Identified Vulnerabilities:

- **Multiplexing Risks:**
  - HTTP/3's independent stream multiplexing can inadvertently expose timing or traffic analysis vulnerabilities.
  - Attackers may exploit stream prioritization mechanisms to infer sensitive data or traffic patterns.
- **Control Frame Exploits:**
  - HTTP/3 relies on control frames, such as `max_table_capacity` and `blocked_streams`, to manage streams and compression.
- **Downgrade Vulnerabilities:**
  - Compatibility with earlier HTTP versions (e.g., HTTP/1.1, HTTP/2) introduces the risk of protocol fallback, exposing legacy vulnerabilities.
- **Privacy Concerns**
  - Features like spin bits, designed for network debugging, can inadvertently leak metadata or enable traffic fingerprinting, compromising user anonymity.

c) Attack Types:

- **DoS Attacks:** HTTP/3 flooding attacks leveraging its unique protocol structure.
- **Slow-rate HTTP/3 POST:** Exploits slow processing of POST requests, causing delays.
- **Tables/Streams Manipulation:** Alters the `max_table_capacity` and `blocked_streams values`, leading to resource exhaustion.
- **Downgrade Attack:** Forces protocol fallback to less secure HTTP versions, increasing the attack surface.
- **Privacy Intrusions:** Break HTTP/3's multiplexing features to analyze user activity patterns.

d) Experimentation and Testbed Setup:

- **Testbed:** Utilized six popular HTTP/3-enabled servers (e.g., OpenLiteSpeed, NGINX, Caddy). Four specific attacks were designed:
- **Flooding:** Overwhelms server resources. This attack increased server response delays up to 3 seconds on average for Cloudflare and caused CPU usage to spike by 99.9% on Caddy within 30 seconds.
- **Slow-rate POST:** Exploits request handling delays. The attack caused delays ranging from 2 to 4 seconds on IIS and Cloudflare servers, with a 10% CPU usage increase.
- **Tables/Streams Manipulation:** Alters control frame parameters to exhaust resources. On NGINX and OpenLiteSpeed, the attack led to temporary paralysis of services, with delays exceeding 10 seconds and unresponsive periods of up to 30 seconds.
- **Downgrade Attack:** Forces the server to revert to HTTP/1.1 or HTTP/2. This attack was highly effective, allowing older protocol vulnerabilities to be exploited and increasing attack surface.
- **Outcome:** Identified vulnerabilities were due to implementation flaws rather than HTTP/3's core design.

e) Key Findings:

- **HTTP/3 Resilience:** Many HTTP/2 vulnerabilities are mitigated in HTTP/3; however, implementation errors still pose risks.
- **Dataset Creation:** "H23Q" dataset includes labeled traffic patterns for 10 distinct attacks, aiding future anomaly detection research.
- **Impact:** Vulnerabilities like CPU spikes and delayed response times were observed under specific attacks, leading to CVE assignments for critical flaws.

f) Mitigations and Defenses:

- **Server Configuration:** Properly disable unneeded protocol support and enforce strict communication policies.
- **Rate Limiting:** Prevent flooding and slow-rate attacks by setting thresholds on request rates.
- **Anomaly Detection:** Deploy machine learning-based systems to identify abnormal traffic patterns.
- **Protocol Updates:** Encourage adoption of patched and secure versions of server software.

g) Conclusions and Implications:

HTTP/3, while addressing several vulnerabilities of HTTP/2, is not immune to security issues, especially due to flawed implementations. The creation of a public dataset enables future research into anomaly detection and defensive mechanisms. Collaborative efforts between researchers and industry are essential to secure HTTP/3 as it becomes the dominant web protocol.

4) The H23Q dataset [5]: Having reviewed the literature and examined various studies on HTTP/3 and QUIC security, we now turn our attention to the dataset employed in this research. The following sections present a comprehensive look at the dataset's structure, the different types of attacks it encompasses, and the results gleaned from both traditional machine learning and deep learning approaches. By analyzing malicious and benign traffic across multiple servers and attack vectors, this dataset provides a rich resource for studying the resilience of HTTP/3 and related protocols under diverse threat scenarios.

This research provides an extensive examination of various attacks against HTTP/3 and QUIC, as well as HTTP/1.1 and HTTP/2, tested on six different servers (Caddy, Cloudflare, OpenLiteSpeed, NGINX, IIS, H2O) under controlled lab conditions. A public dataset was created, encompassing a wide range of attacks, normal (benign) traffic, and associated metadata (features) to enable machine-learning-based analysis and attack detection. Below are the main findings:

a) Attacks: **HTTP/3 Flooding Attack**

- *Description*: Sending multiple HTTP/3 requests in short bursts (parallel `curl` processes).
- *Impact*:
  - Caddy: CPU usage soared to 99.9% within 30 seconds, effectively paralyzing the server.
  - Cloudflare: Response delay reached about 3 seconds after 30 seconds of attack.
  - Other servers: Minimal CPU increase ($< 5\%$).

HTTP/x Downgrade Attack

- *Description*: Exploits backward compatibility, downgrading HTTP/3 to HTTP/1.1 or HTTP/2 via TCP connections.
- *Impact*:
  - All tested servers allowed downgrade.
  - Enlarged the overall attack surface.

Slow-Rate HTTP/3 POST Attack

- *Description*: 40 parallel POST requests with payloads generated by OpenSSL, each connection dropped after 5 seconds.
- *Impact*:
  - Caddy: CPU usage spikes, causing delayed responses.
  - Other servers largely unaffected.

HTTP-Tables/Streams Attack

- *Description*: Extreme manipulation of `max_table_capacity` and `blocked_streams` values (very small vs. very large).
- *Impact*:
  - Caddy: CPU reached 99.9% within seconds.
  - H2O: Response delays exceeded 4 seconds.
  - NGINX & OpenLiteSpeed: Became unresponsive for up to 30 seconds.
  - Cloudflare: Minimal delay, showing strong resilience.

b) Dataset Structure:

- **Files**: 60 `.pcap` files (10 attacks $\times$ 6 servers).
- **10 Types of Attacks**:
  1) HTTP/3 Flood
  2) Fuzzing
  3) HTTP/3 Loris
  4) HTTP/3 Stream
  5) QUIC Flood
  6) QUIC Loris
  7) QUIC Encapsulation
  8) HTTP Smuggling
  9) HTTP/2 Concurrent
  10) HTTP/2 Pause-Resume
- **Key Statistics**: Malicious traffic ranges from 0.08% (HTTP/3 Stream) to $\sim$38% (HTTP/3 Flood) of total traffic.
- **Formats**: `.pcap` and `.csv` (labeled with $\sim$ 200 features); TLS keys are provided for QUIC traffic decryption.

c) Attack Signatures:

- **HTTP/3 Flood**: Linear increase with spikes; $\sim 10\text{k}-15\text{k}$ frames/s at peak.
- **Fuzzing**: Irregular peaks; $\sim 8\text{k}-10\text{k}$ frames/s on average.
- **HTTP/3 Loris**: "Plateau" caused by long-held connections; $\sim 4\text{k}-6\text{k}$ frames/s.
- **HTTP/3 Stream**: Smooth rise with consistent peaks.
- **QUIC Flood**: Rapid increase in the first 2 minutes; $\sim 14\text{k}-18\text{k}$ frames/s.
- **QUIC Loris**: Gradual increase, then a plateau at high load; $\sim 7\text{k}-8\text{k}$ frames/s.
- **QUIC Encapsulation**: Bursty patterns ($\sim 5\text{k}-7\text{k}$ frames/s).
- **HTTP Smuggling**: Quick spikes followed by moderate leveling.
- **HTTP/2 Concurrent**: Linear buildup during peak traffic ($11\text{k}-13\text{k}$ frames/s).
- **HTTP/2 Pause-Resume**: Intermittent drops ($3\text{k}-5\text{k}$ frames/s) during pause phases, climbing to $\sim 10\text{k}$ during resumes.

d) Dataset Quality Assessment:

- **Class Distribution**:
  - Normal: 50%
  - DDoS-Flooding: 20%
  - DDoS-Loris: 15%
  - Transport-Layer: 10%
  - HTTP/2 Attacks: 5%
- **Data Sources**: Mix of real (normal) traffic and simulated attacks.
- **Supplementary Tools**: TLS keys for QUIC traffic analysis.

e) Experiments:

- **Testbed**:
  - Servers: OpenLiteSpeed, Caddy, NGINX, H2O, IIS, Cloudflare.
  - Clients: 13 devices across three subnets.
- **Classes Evaluated**: Normal traffic, DDoS-Flooding, DDoS-Loris, Transport-Layer attacks, and HTTP/2-based attacks.

f) Classification Analysis (Supervised Learning): **Shallow Classification (Random Forest, Logistic Regression)**

- Overall Accuracy:
  - Normal: 96%
  - DDoS-Flooding: 92%
  - DDoS-Loris: 87%
  - Transport-Layer: 85%
  - HTTP/2 Attacks: 80%
- **Key Features**: Packet size, inter-arrival time, stream IDs.

Deep Neural Network (DNN) (3-layer model)

- Overall Accuracy:
  - Normal: 98.5%
  - DDoS-Flooding: 97%
  - DDoS-Loris: 93%
  - Transport-Layer: 91%
  - HTTP/2 Attacks: 89%
- **Crucial Features**: Frame counts, entropy, protocol headers.

g) Anomaly Detection:

- **Results**:
  - DDoS-Flooding (HTTP/3 Flood, QUIC Flood) showed high anomaly scores with low false positives.

  - DDoS-Loris and Transport-Layer attacks overlapped somewhat with normal traffic patterns.
  - HTTP/2 Pause-Resume displayed a clear signature at pause-transition intervals.

h) Overall Takeaways:

- The research demonstrates multiple ways to exploit weaknesses in HTTP/3/QUIC (and older HTTP versions) via flooding, slow-loris–style attacks, transport-layer manipulations (encapsulation/smuggling), and other engineered traffic anomalies.
- A **publicly available dataset** has been created, featuring `.pcap` and `.csv` files (labeled with $\sim$ 200 features), along with TLS decryption keys for QUIC data.
- Classification approaches (both shallow and deep) achieve solid detection rates (80–98.5% accuracy) for different attack types based on network and protocol-level attributes.
- Some protective solutions—like Cloudflare—show stronger resilience, whereas mainstream servers (Caddy, NGINX, OpenLiteSpeed) exhibit high CPU spikes and service disruption under heavy attack scenarios.

In summary, the dataset serves as a valuable resource for assessing the security posture of HTTP/3 and QUIC under varied threat models, highlighting the importance of ongoing research and adaptive defense mechanisms.

5) A Survey on the Security Issues of QUIC [6]: The article focuses on a comprehensive review of the QUIC protocol (Quick UDP Internet Connections) and examines the security challenges arising from its use. Developed by Google and standardized as RFC 9000 in 2021, QUIC is considered a significant advancement in reducing latency and improving communication performance compared to TCP, particularly in HTTP/3 communications.

a) Research Objectives:

- To understand the operational principles of QUIC.
- To identify existing security threats, such as Replay Attacks, DDoS attacks, and Cache Poisoning.
- To propose solutions and areas for future research.

b) Key Findings:

- **Improvements:** The QUIC protocol includes unique features such as Connection Migration and Zero Round Trip Time (0-RTT), which enhance user experience.
- **Challenges:** The use of UDP exposes the protocol to vulnerabilities such as Spoofing and Reflection Amplification attacks.

c) Comparison to Other Attacks: QUIC-specific attacks present unique risks compared to TCP, such as the *Stateless Reset* mechanism, which can enable Denial-of-Service (DoS) attacks.

d) Examples of Attacks Discussed in the Article:

- **Cache Poisoning:** Attackers inject fake data into DNS or HTTP caches, redirecting users to malicious content.
- **Reflective DDoS Attacks:** Exploiting server responses to flood a victim with massive traffic.
- **Slowloris:** Utilizing slow connections to deny legitimate users access.
- **Fake ACK Attacks:** Disrupting load control by exploiting acknowledgment mechanisms.

e) Recommendations and Mitigation Measures:

- Use authentication mechanisms such as DNSSEC and cryptographic validation methods.
- Develop tools for real-time threat monitoring and detection.
- Optimize defenses to handle mechanisms like 0-RTT without compromising security.

f) Conclusions and Implications:

QUIC offers numerous advantages over TCP but also introduces unique security challenges that require innovative solutions. The article emphasizes the need for advanced and efficient defenses, particularly against sophisticated attacks that exploit weaknesses in the protocol.

6) An Empirical Approach to Evaluate the Resilience of QUIC Protocol Against Handshake Flood Attacks [7]: The article analyzes the vulnerabilities of the QUIC protocol under Handshake Flood attacks, which aim to crash servers by exploiting the protocol's handshake mechanism. The research focuses on comparing QUIC with TCP Syn Cookies to evaluate their resilience against DDoS attacks and the implications of QUIC's built-in authentication mechanism on server resources.

a) Research Objectives:

- To examine server resource consumption under Handshake Flood attacks, including CPU, memory, and bandwidth.
- To measure the amplification factor of DDoS attacks in QUIC and compare it to TCP Syn Cookies.
- To identify QUIC's key weaknesses and provide insights for future protocol development.

b) Vulnerabilities:

- **High Amplification Factor:** QUIC servers exhibit an amplification factor of 4.6, making them more vulnerable compared to TCP Syn Cookies (1.28).
- **CPU Resource Bottleneck:** QUIC servers experience very high CPU usage under attack, leading to service crashes.
- **Complex Protocol Design:** The authentication and packet creation process in QUIC requires extensive computations, unlike the simpler TCP protocol.

c) Attacks Types:

- **Handshake Flood Attack:** Exploits the handshake process to overload server resources.
- **Spoofed Handshake Packets:** Sends fake requests, forcing the server to allocate resources without completing the handshake process.

d) Experimentation and Testing:

- **Experimental Setup:**
  - **Server:** Raspberry Pi (1GB RAM, Quad-Core 1.2GHz).
  - **Client:** Laptop (16GB RAM, Intel i7 1.8GHz).

  - **Validation:** Server and client roles were swapped to ensure reliable results.
- **Measured Resources:**
  - CPU
  - Memory
  - Bandwidth
- **Protocols Tested:**
  - QUIC (`aioquic`)
  - TCP Syn Cookies

e) Key Findings:

- **CPU Usage:** QUIC servers reached 73% CPU usage at 50 packets per second, compared to less than 1% in TCP Syn Cookies.
- **Memory:** QUIC's memory consumption increased gradually but was not the limiting factor.
- **Bandwidth:** Both protocols exhibited symmetric bandwidth usage, reducing exposure to reflection attacks.
- **Amplification Factor:**
  - **QUIC:** 4.6
  - **TCP Syn Cookies:** 1.28

f) Mitigation Strategies:

- **Rate Limiting:** Limit packet rates to prevent server overload.
- **Retry Packets:** Use Retry packets to verify client addresses before allocating additional resources.
- **Protocol Improvements:**
  - Simplify the handshake process.
  - Optimize authentication mechanisms.

g) Conclusions and Implications:

The research highlights significant vulnerabilities in QUIC's design, particularly under Handshake Flood attacks. CPU usage is the primary bottleneck, while bandwidth and memory are not limiting factors. Although QUIC offers improved performance over TCP, these vulnerabilities underscore the need for further optimization and dedicated defense mechanisms.

7) Manipulated Client Initial Attack and Defense of QUIC [9]: The article presents a novel Denial-of-Service (DoS) attack targeting the QUIC protocol, leveraging manipulated client initial (MCI) packets. The attack exploits QUIC's handshake mechanism to overload server resources. The research aims to introduce the attack, measure its impact, and propose defense strategies, including the use of hardware acceleration mechanisms.

a) Research Objectives:

- To identify vulnerabilities in the QUIC protocol under DoS scenarios focused on its encryption and handshake mechanisms.
- To analyze the impact of MCI attacks on various QUIC implementations.
- To propose hardware-based solutions to mitigate the effects of DoS attacks.

b) Vulnerabilities:

- **Impact of MCI Attacks:** Manipulated client initial packets exploit QUIC's need for authentication and encryption, causing excessive CPU resource usage. The decryption and validation processes result in significant resource consumption on servers.
- **Vulnerable QUIC Implementations:** All five tested QUIC implementations (picoQUIC, Quicly, MsQuic, Nginx, NGTCP2) exhibited severe performance degradation under attack.

c) Attack Types: **Manipulated Client Initial (MCI) Attack:** Sending spoofed initial packets to exhaust server resources. The attack targets QUIC's cryptographic requirements during the handshake process.

d) Experimentation and Testing:

- **Experimental Setup:** Tests were conducted on five active QUIC implementations under controlled lab conditions. Scenarios included both normal traffic and MCI-induced traffic loads.
- **Metrics Evaluated:** Bandwidth performance, CPU consumption, and successfully processed packets per second.
- **Solution Comparisons** Performance analysis of Address Validation mechanisms versus Inline Offloading.

e) Key Findings:

- **Performance Impact:** MCI attacks reduced the average throughput per core by 85% compared to normal traffic. QUIC implementations under attack performed at only 15% of their standard traffic capacity.
- **Hardware-Based Solutions:** Inline Offloading improved server resilience, achieving a processing rate of up to 67Gbps. Address Validation reduced cryptographic CPU consumption by approximately 45%.

f) Mitigation Strategies:

- **Address Validation:** Using Retry Packets to authenticate traffic sources before further processing.
- **Inline Offloading:** Accelerating packet processing using Network Acceleration Complex (NAC). Enabled processing of up to 192,000 packets per second per core.

g) Conclusions and Implications:

The research highlights weaknesses in QUIC's handshake and encryption mechanisms under MCI attacks. Hardware solutions like Inline Offloading offer an efficient way to mitigate these attacks while maintaining high performance. The study emphasizes the need for continued development of secure protocols and advanced technologies to counter emerging threats.

8) QUICLORIS - A Slow Denial-of-Service Attack on the QUIC Protocol [8]: The article analyzes the vulnerabilities of the QUIC protocol to slow-rate Denial-of-Service (Slowloris) attacks, emphasizing the use of this advanced transport layer protocol in HTTP/3 infrastructure. The research demonstrates how QUIC's built-in mechanisms, such as PING frames and flow control, can be exploited to exhaust server resources.

a) Research Objectives:

- To evaluate the vulnerabilities of the QUIC protocol to slow-rate attacks like Slowloris.
- To examine the impact of similar attacks, such as Ping Flood and amplification-based attacks.
- To propose defensive measures to prevent these attacks.

b) Key Vulnerabilities:

- Using PING frames to prolong connections beyond their necessary duration.
- Manipulating flow control mechanisms to overload server resources.
- Opening multiple connections with partial requests, leading to thread exhaustion.

c) Attacks Types:

- **Slowloris Attack:** Opening multiple connections and sending slow requests to prevent connection termination.
- **Ping Flood:** Overwhelming servers with ICMP packets to disrupt their responsiveness.
- **Flow Control Exploit:** Exploiting flow control windows to overload server resources.
- **Replay and Fragmentation Attacks:** Compromising data integrity and disrupting data transmission.
- **Amplification Attacks:** Using spoofed packets to generate high server load.
- **R.U.D.Y (R U Dead Yet):** Sending extremely slow data streams to block access for legitimate users.
- **Version Negotiation Exploit:** Forging messages during QUIC's version negotiation stage to cause incompatibility and communication failure.

d) Experimentation and Results: Experiments were conducted in a controlled environment using the aioquic tool to evaluate the attacks.

e) Key Findings:

- Complete server shutdowns occurred due to active but non-functional connections.
- Significant increases in CPU and memory usage during Slowloris and Ping Flood attacks.
- Response delays and reduced system throughput following Replay and Fragmentation Attacks.
- High susceptibility to Amplification and Version Negotiation Exploit attacks.

f) Conclusions and Implications:

The handshake phase of the QUIC protocol is particularly vulnerable to slow-rate attacks and must be fortified. Recommended measures include filtering spoofed traffic and detecting suspicious behavior, limiting the use of PING frames and monitoring unusual request patterns, and implementing adaptive flow control for efficient resource allocation. While the QUIC protocol is suitable for general use, it requires further adjustments and enhanced security for critical environments, such as financial systems.

9) 0-RTT Attack and Defense of QUIC Protocol [10]: The article analyzes the security vulnerabilities of the QUIC protocol, a transport layer protocol developed by Google and currently serving as the foundation for HTTP/3. The research primarily focuses on the connection establishment phase of the protocol and introduces a novel attack called the "0-RTT Attack," which examines attackers' ability to exploit security weaknesses in QUIC's connection setup mechanism.

a) Research Objective:

- To identify security weaknesses in QUIC.
- To demonstrate an effective attack capable of disrupting connections between clients and servers.

b) Main Vulnerabilities:

- Unencrypted initial handshake messages make them susceptible to forgery.
- The connection-oriented nature of UDP allows exploitation through spoofed packets.

c) Attack Types:

- **QUIC RST Attack:** Forging connection reset packets to force the client to terminate the connection.
- **Version Negotiation Spoofing Attack:** Sending fake version negotiation packets to cause the client to disconnect.

d) Experimental Methods and Results: The experiment was conducted in a local area network (LAN) using techniques such as ARP Spoofing and packet capture. The attacks successfully caused clients to terminate connections due to spoofed packets, focusing on vulnerabilities in various versions of QUIC, including the latest tested version (Q041).

e) Conclusions and Recommendations: QUIC's security mechanisms are robust post-connection, but the connection setup phase remains vulnerable. Suggested mitigation measures include implementing a short delay for packet verification and introducing multiple validation checks during the connection setup phase. QUIC is not recommended for use in critical environments, such as financial trading, due to these weaknesses.

f) Broader Implications:

Further research is needed to integrate security solutions without compromising the protocol's efficiency and speed. Users of the protocol should exercise caution when employing QUIC in scenarios requiring high-security levels.

H. Contribution

The current project focuses on developing and demonstrating a new or existing attack on the HTTP/3 protocol (based on QUIC) while simultaneously proposing a dedicated defense mechanism for the attack. Compared to previous works, which primarily analyzed known attacks (such as the Slowloris Attack, Ping Flood, or flow control-based attacks) and adapted general defenses, our work emphasizes the development and implementation of an innovative attack that exploits an existing weakness in the HTTP/3/QUIC protocol but has not yet received thorough attention in the research literature.

In this study, we aim to:

- **Define a new vulnerability in the HTTP/3/QUIC protocol:** Conduct an in-depth analysis of the Handshake mechanisms, connection management, and UDP characteristics in the QUIC protocol to identify a "loophole" for an attack that has not been addressed in existing research.
- **Develop a proof of concept for the attack:** Design a demonstration scenario and execute the attack in a controlled environment to validate its impact on server/service availability, resource consumption, or end-user experience.

- **Propose an innovative defense mechanism:** Develop a dedicated defense mechanism based on monitoring unique parameters of HTTP/3/QUIC to effectively address this specific attack and protect servers in real-world environments.
- **Evaluate performance and effectiveness:** Test the attack and the defense mechanism across a variety of practical scenarios and network parameters (e.g., bandwidth, latency, packet loss, and NAT configurations) to provide a comparative analysis against known attacks and defenses. This evaluation will characterize the attack's severity and the effectiveness of the proposed defense mechanism.

The primary contribution of this research lies in the dual development of a new/unmitigated attack and an innovative dedicated defense. On one hand, we expand the attack surface by identifying a unique and underexplored vulnerability. On the other hand, we propose a defense mechanism specifically tailored to address this vulnerability. This approach allows us to highlight the limitations of existing solutions and provide a practical, field-ready solution in both design and implementation.

As a result, we hope this research will advance the understanding of security risks in the HTTP/3/QUIC protocol and encourage the development of more robust defense mechanisms against future attacks.

I. Background on QUIC

This section covers QUIC packet structure, header protection, AEAD encryption, ACK frame format, and loss recovery behaviors referenced by our attack. We rely on RFC 9000/9001 terminology and the practical implementation patterns common in `aioquic`/`ngtcp2`.

1) QUIC v1 vs v2 — Key Differences: Table I summarizes the protocol-level changes introduced in QUIC v2 with the goal of mitigating ossification and strengthening downgrade and path validation protections while keeping the HTTP/3 API surface intact.

II. Related Work (Threat Landscape)

In this section we consolidate findings from the literature regarding known attacks on HTTP/3 and QUIC, and provide a structured overview of the threat landscape. To complement the textual survey, we include two comprehensive tables that summarize the main attack vectors, their characteristics, and their qualitative assessments.

Table II presents a survey of documented and experimentally demonstrated attacks targeting HTTP/3 and QUIC. It lists for each attack its core mechanism, relevant references, and known countermeasures. This tabular representation allows the reader to quickly grasp which types of attacks have been practically validated in prior work, and how they affect the protocol stack.

Table III extends this view by mapping broader attack scenarios and providing a qualitative assessment of their severity and feasibility. This table synthesizes insights drawn from the broader survey literature [6], offering a threat-centric classification that highlights not only the attacks implemented in practice, but also those theorized or anticipated by the research community. By situating each scenario along dimensions of protocol layer, evaluation context, and severity, this mapping supports a more holistic understanding of the evolving attack surface.

A. QUICforge: Client-side Request Forgery in QUIC

QUICforge (NDSS 2023) systematizes client-side request forgery against QUIC and demonstrates three forgery families that exploit unencrypted/observable fields during specific stages of a QUIC connection:

- **SIRF (Server-Initial Request Forgery)** — targets the Initial/Retry stage by manipulating fields like *DCID* (up to 20 bytes), *Token*, and *Packet Number*, causing Initial/Retry packets to be reflected to a spoofed victim and potentially disrupting address validation and TLS 1.3 progress.
- **CMRF (Connection-Migration Request Forgery)** — abuses path validation (`PATH_CHALLENGE`/`PATH_RESPONSE`). By forging *SCID/DCID* with a new 5-tuple (IP/port), the server emits `PATH_RESPONSE` to the victim and may tear down the legitimate path.
- **VNRF (Version-Negotiation Request Forgery)** — crafts oversized Version Negotiation (VN) packets by leveraging an *extended CID* (up to 256 bytes) and an attacker-

TABLE I
Main Differences Between QUIC v1 and v2

Topic	v1	v2	Purpose
Version Number	0x00000001 (fixed)	Pseudo-random value	Prevent middlebox ossification.
Packet Type Codes	Fixed mapping	New mapping	Break detection patterns.
Initial Salt	Fixed	New per-version salt	Avoid predictable keys.
TLS Label Prefixes	``quic v1''	``quic v2''	Prevent confusion across versions.
Retry Integrity Tag	Fixed key/nonce	New values	Preserve integrity for v2 Retry.
Version Negotiation	Unencrypted	Encrypted (RFC 9368)	Downgrade resistance.
Handshake Security	TLS 1.3 (initial keys)	TLS 1.3 (different initial keys)	Preserve connection integrity.
CID Management	Fixed CID across path	CID replacement recommended	Reduce tracking across paths.
Stateless Reset	Fixed token	Per-CID token	Safe connection termination.

TABLE II
SURVEY OF DOCUMENTED ATTACKS ON HTTP/3 AND QUIC

Name	Description	Paper	Prevent/Reduce	Proto.	Code	Dataset
HTTP/3 flooding attack	Overwhelms the server by sending numerous HTTP/3 requests in parallel, consuming CPU and bandwidth.	*A hands-on gaze on HTTP/3 security through the lens of HTTP/2 and a public dataset*	Rate limiting, prioritization, traffic shaping	HTTP/3	https://github.com/efchatz/HTTP3-attacks	H23Q
HTTP/x downgrade attack	Forces downgrade from HTTP/3 to older versions, exposing legacy vulnerabilities.	*A hands-on gaze on HTTP/3 security through the lens of HTTP/2 and a public dataset*	Disable unused versions, enforce ALPN	HTTP/3	Script (verify)	H23Q
Slow-rate HTTP/3 POST attack	Sends POST bodies slowly to tie up server resources.	*A hands-on gaze on HTTP/3 security through the lens of HTTP/2 and a public dataset*	Timeouts, min bandwidth	HTTP/3	https://github.com/efchatz/HTTP3-attacks	H23Q
HTTP-tables/streams attack	Abuses QPACK table capacity and blocked streams to exhaust memory.	*A hands-on gaze on HTTP/3 security through the lens of HTTP/2 and a public dataset*	Limit table size, cap streams	HTTP/3	https://github.com/efchatz/HTTP3-attacks	H23Q
DoS via Connection Migration	Triggers repeated migration to overload NATs/load balancers.	*Security and Service Vulnerabilities with HTTP/3*	Limit migration frequency	QUIC	https://github.com/hari-19/http3-vulnerability-analys	—
Covert Channel (spin bit)	Uses spin bit for covert data exfiltration.	*Security and Service Vulnerabilities with HTTP/3*	Limit spin bit usage	QUIC	https://github.com/hari-19/http3-vulnerability-analys	—
0-RTT Attack	Abuses early data to replay/disrupt connections.	*0-RTT Attack and Defense of QUIC Protocol*	Anti-replay checks	QUIC	—	—
Handshake Flood Attacks	Overwhelms CPUs via handshake abuse.	*An Empirical Approach to Evaluate the Resilience of QUIC Protocol Against Handshake Flood Attacks*	Stateless retry, rate limiting	QUIC	—	—

chosen Version value; the server reflects a large VN packet (up to the Initial-size budget, e.g., $\sim$1200 B) to the spoofed destination, enabling cross-protocol interactions (e.g., DNS) and potential DoS amplification.

The authors discuss anti-amplification and address-validation constraints but also show that pre-handshake and migration-time behaviors can still be steered to third-party addresses. Their proof-of-concept code underpins our exploration of ACK-centric reflection.

B. How QUICforge informs our Spoofed-ACK design

Our attack builds upon the QUICforge insight that selectively controllable, partially visible handshake/migration elements can be exploited to reflect traffic to arbitrary endpoints. We extend this idea to ACK-driven loss recovery: by forging credible ACK frames post-handshake (1-RTT) or under 0-RTT reuse, we aim to stimulate server retransmissions or loss-probing bursts toward a spoofed target, or to degrade a legitimate flow via unnecessary recovery.

C. Phase-wise Early Exploration

a) Phase 1 — Downgrade Attack (reproduction).: We began by reproducing the HTTP/x Downgrade described in [5]. The attack coerces servers to fall back from HTTP/3 to HTTP/2 or HTTP/1.1, re-exposing legacy weaknesses. In the paper, all six evaluated servers accepted downgrade, signaling a broad exposure. Because no code was provided, we authored a script that emulates a client–server session across HTTP/3/HTTP/2/HTTP/1.1 and examined feasibility on QUIC v2. Although RFC updates mitigate *QUIC-version* downgrade, we observed that *HTTP/3*-level downgrade remains possible under v2, preserving the risk.

b) Phase 2 — Feasibility via QUICforge and ACK-centric design.: Next, we evaluated request-forgery surfaces in QUIC with an emphasis on coupling Spoofed ACKs to DoS-style amplification. Guided by QUICforge, we analyzed SIRF/CMRF/VNRF on both QUIC v1 and v2 and identified points where attacker-crafted ACK frames could plausibly trigger excessive retransmissions or recovery bursts. A key ob-

TABLE III
ATTACK SCENARIOS AND QUALITATIVE ASSESSMENT

Name	Description	Proto.	Evaluation / Notes
Reflection-based DDoS	Spoofed requests trigger large server responses.	QUIC	Severe; possible early in handshake.
Handshake Tampering	Alters handshake messages to disrupt sessions.	QUIC	Severe; needs strong validation.
Spoofed ACK Attack	Forges ACKs to cause excessive retransmissions.	QUIC	Severe; feasible if handshake is complete.
Slowloris Attack	Keeps many idle connections open.	HTTP/3/QUIC	Moderate; mitigations known.
Cache Poisoning	Injects false data into caches.	—	Severe; depends on cache logic.
Stream Fragmentation	Sends partial fragments to degrade service.	QUIC	Moderate; handshake-adjacent.
ECN Abuse	Manipulates ECN to disrupt congestion control.	QUIC	Severe; needs research.
Optimistic ACK Attack	Sends premature ACKs.	QUIC	Low severity; mitigated by strict checks.
Reflective Amplification	Server amplifies traffic to victim.	QUIC	Severe; handshake protection helps.
Firewall Negligence	Firewalls fail to inspect QUIC traffic.	—	Low severity; operational issue.
DoS via Connection Migration	Repeated migrations overload state.	QUIC	Variable severity; CID tracking mitigates.

servation from QUICforge is the allowance of up to 256 bytes for the CID in version negotiation, much larger than in other stages; while the paper demonstrated cross-protocol DNS spoofing and noted the possibility of DoS amplification, it did not quantify it—leaving an opportunity we pursue in our design and evaluation.

D. QUICForge (NDSS 2023): Client-side Request Forgery in QUIC

QUICForge systematically demonstrates client-side request forgery techniques against QUIC by exploiting fields that remain partly visible or controllable during early connection stages. The paper presents proof-of-concept code and three concrete forgeries—SIRF, CMRF, and VNRF—and evaluates them on mainstream QUIC stacks. Our Spoofed-ACK attack *builds upon* QUICForge: we adopt its tooling and environment, but pivot the forgery towards manipulating `ACK` frames to *trigger bursty retransmissions* and reflection-like effects rather than only causing desynchronization.

Key observation.: Unlike the other forgeries, VNRF allows *extended CIDs* up to 256 bytes, which QUICForge used for cross-protocol DNS spoofing. The paper hypothesizes amplification but does not quantify it—a gap our work addresses by measuring retransmission bursts induced by forged `ACK` frames.

E. Phase-wise Early Exploration

Phase 1: HTTP/x Downgrade Attack. We reproduced the downgrade attack from [5], coercing servers to fall back from HTTP/3 to HTTP/2 or HTTP/1.1 and thus re-exposing legacy weaknesses. All six servers in the paper accepted downgrades. Because no code was provided, we built a small script to simulate a server–client session across HTTP/3/2/1.1. Reading the most recent RFCs revealed mitigations for *QUIC* version downgrade in v2, yet these do not preclude *HTTP/3* downgrades; thus the attack surface persists in practice.

Phase 2: Feasibility via QUICForge, ACK-centric design. We evaluated QUICForge's code on both v1 and v2, and identified injection points to craft malicious `ACK` frames so that the server marks ranges as missing and retransmits N packets—a basis for a reflection/depletion DoS.

F. Reproducing Prior Work: Infrastructure and Workflow

Two-tier Docker architecture.

- **HTTP/3 server:** `aioquic` (HEAD Mar 2025), OpenSSL/BoringSSL, TLS 1.3 dependencies [12].
- **Attacker host:** `aioquic`, Scapy 2.5.0, NetfilterQueue 1.2, BoringSSL, `iptables-NFQUEUE` [12].

Each VM/container used a distinct IP (no localhost dependencies) to faithfully demonstrate cross-host attacks.

Workflow.

1) Re-run QUICForge code on v1; collect PCAPs and key logs.
2) Adapt to v2 (update Version field constants).
3) Re-run and analyze with Wireshark/QVIS/QLOG.
4) Compare v1 vs v2 behaviors (no functional divergence observed for the studied paths).

TABLE IV
QUICFORGE FORGERY SCENARIOS AND IMMEDIATE EFFECTS

Scenario	Attacked Stage	Attacked plaintext fields	Immediate effect	Notes / security implications
SIRF (Server-Initial Request Forgery)	Initial/Retry	DCID (20B), Token, Packet Number	Initial/Retry packet reflected to victim; handshake may stall or fail	Interferes with address validation; interacts with TLS 1.3 onboarding.
CMRF (Connection-Migration Request Forgery)	Path validation	SCID/DCID and new 5-tuple (IP/port)	PATH_RESPONSE sent to victim; legitimate path may be torn down	Enables hijack/desync, bypasses naive path filters.
VNRF (Version-Negotiation Request Forgery)	Version negotiation	Version field, extended CIDs (up to 256B)	Large VN packet (up to 1200B) reflected to victim	Supports cross-protocol tricks (e.g., DNS spoofing); potential amplification.

G. Technical Challenges and Fixes

TABLE V
ENGINEERING CHALLENGES WHILE REPRODUCING QUICFORGE AND OUR FIXES

Issue	Description	Fix
Out-of-date diff	Provided `.patch` did not match current `aioquic`	Manual re-implementation of changes and a refreshed diff.
Missing run docs	Conflicting library versions and no canonical command sequence	Authored a `Makefile` and execution logs with pinned versions.
QUIC ≡ UDP in Wireshark	Port not auto-decoded as QUIC	Added *Decode-As* rule and `tls-keylog-file`
NFQUEUE in Docker	Dropped privileges block `IPTables-NFQUEUE`	Run with `--cap-add=NET_ADMIN,NET_RAW --privileged`.
Cross-protocol DNS	VN-based DNS spoofing yielded partial responses only	Abandoned this path in favor of Spoofed-ACK design.

H. Derived Threat Model (from reproduction)

The attacker holds (or acquires) an Address Validation token (leak/phishing/insider), sets up a legitimate connection to learn parameters and collect TLS secrets, intercepts client `ACK`s, decrypts and edits `ACK Range`s and gaps to mark N packets as "missing", then re-encrypts and sends them to the server so it retransmits those N packets—optionally towards a third-party victim IP (reflection).

I. Key Findings from Prior-Work Reproduction

- Request-forgery attacks remain effective on QUIC v2.
- Spoofed-ACK requires QUIC-layer access (header parsing, authenticated re-encryption); UDP 5-tuple spoofing alone is insufficient.
- We identify a structural weak point (the `ACK Range`) that enables *flood-triggered retransmissions*.
- Next step: implement an *on-path re-encrypt* module in `aioquic` for dynamic `ACK`-frame edits and instrument server-load metrics.

III. THREAT MODEL AND ASSUMPTIONS

Adversary. A network-adjacent or on-path attacker capable of packet capture and modification on the client–server path (e.g., same L2 segment, transparent bridge, or NAT box).
Capabilities. Reads TLS secrets from a colocated client (shared `sslkeylogfile`), leverages NetfilterQueue to intercept/modify UDP/QUIC packets, and forges source IP/port tuples.
Goals. Trigger excessive retransmissions or loss-recovery behaviors at the server towards a spoofed target (reflection) or degrade the legitimate connection (depletion).
Assumptions. Handshake has completed (1-RTT established) or 0-RTT re-use exists; address validation satisfied; anti-amplification limits lifted for the path.

IV. APPROACH / ATTACK DESIGN (OUR METHOD)

We implement a four-phase Spoofed-ACK pipeline using `aioquic`, `scapy`, and `netfilterqueue`. A high-level schematic illustrates interception, decryption, ACK rewriting, and re-encryption.

A. Phase 3 – Attack Strategy and Pseudo-Algorithm

1) Objective and gap vs. QUICForge: QUICForge confines forgery to IP/port and the version field at configuration time; it does not alter QUIC packets themselves. Our goal is to extend this to a *Spoofed-ACK* attack that (i) intercepts an in-flight QUIC packet, (ii) decrypts an `ACK` frame, (iii) rewrites packet-number ranges, and (iv) re-encrypts the packet without breaking the connection.

2) ACK-frame vulnerability (core insight): Unlike TCP, QUIC `ACK` frames encode a *set of ranges*. Inserting an artificial "hole" (e.g., declaring packets $[1000, 5000]$ as missing) induces a retransmission burst of thousands of packets. This yields: (1) *reflection* (server answers a spoofed IP) and (2) *load amplification* (DoS via retransmissions).

TABLE VI
PHASE-3 PSEUDO-ALGORITHM FOR SPOOFED-ACK.

Step	Action	Outcome / Goal
1	Launch a legitimate H3 client container and open a clean session.	Session Ticket and SSL key log written to a shared volume.
2	Attacker opens a 0-RTT connection using the shared ticket.	Fast authenticated setup (no full handshake).
3	Attach `NFQUEUE` to intercept client→server UDP/QUIC.	Real-time capture.
4	Identify target QUIC packets and extract candidate `ACK` frames.	Candidates selected.
5	Decrypt → rewrite `ACK` ranges (forge holes) → re-encrypt.	Server believes many packets are missing.
6	Redirect reflected traffic to a fake "DB" victim (distinct IP).	Collect retransmissions.
7	Run the three containers together to validate end-to-end behavior.	Attacker triggers spoofed ACKs; victim receives bursts.

3) Design principles: After discussion with our advisor we avoided crafting brand-new packets (high risk of malformed/illegal frames). Instead we *modify a legitimate packet in place*: *"Take what the protocol already trusts and twist it."* We split the work along two axes: (i) infrastructure — a three-container Docker lab (client, attacker, fake DB), and (ii) cryptography — in-place decrypt/modify/re-encrypt of captured QUIC packets.

4) Pseudo-algorithm (high level):

B. Implementation Highlights and Code Evolution

Below we summarize only the core functions (with figures pointing to the relevant code excerpts), describing their role in the attack pipeline without listing entire files.

1) `ack_callback(packet, args=None):` Intercepts QUIC packets via NFQUEUE, filters by 4-tuple and connection IDs, parses packet number space, and invokes decrypt/modify/re-encrypt routines.

2) `configure_ack_client(args):` Loads `sslkeylogfile`, derives initial/handshake/1-RTT secrets, configures socket hooks and NFQUEUE bindings, and initializes QUIC connection context.

3) `decrypt_quic_packet(packet, keylog_path):` Removes header protection, reconstructs sample, and performs AEAD open to obtain the plaintext payload and parsed header. Used prior to any frame manipulation.

4) `spoof_ack_packet(packet, args, modifier_func):` Given a decrypted payload, synthesizes ACK frames targeting chosen packet number ranges; for reflection, the source tuple is re-forged to the victim.

5) `modify_ack_frames(payload, header, pn):` Edits ACK ranges (`largest_acknowledged`, `ack_delay`, blocks) to bias server loss recovery and cause retransmissions.

6) `_encode_uint_var(value):` Ensures RFC-compliant QUIC varint encoding when (re)building frames after edits.

7) `reencrypt_quic_packets(packet, secrets, decrypted_packets):` Re-applies header protection and AEAD seal to produce valid ciphertext after manipulation.

C. Status and Continuation Plan

V. EXPERIMENTAL SETUP (METHODOLOGY)

1) Docker Testbed and Roles: We deploy a three-node Docker testbed with distinct IPs (no `localhost` dependencies) to demonstrate cross-host behavior.

Note. Container capabilities: `NET_ADMIN`, `NET_RAW`, `SYS_PTRACE` were required for NFQUEUE/raw-socket interception.

Note. The HTTP/3 server container was prepared earlier and is therefore omitted from Table VIII.

2) Stage A — Legitimate Connection and Session Ticket Sharing (3.5): A valid HTTP/3 session is initiated by `client-legit`. The SSL key log is persisted but used later only for diagnostic verification; the session ticket is written to a shared volume for subsequent 0-RTT re-use.

3) Stage B — 0-RTT: Failure and Resolution (3.6): Attempt #1. Using only the SSL key log yielded a regular 1-RTT handshake; 0-RTT was rejected. *Investigation.* The RFCs and `aioquic` reveal that keys may change/roll; 0-RTT requires reusing a valid *session ticket*. We adapted the QUICforge code to persist the ticket into the shared volume. *Attempt #2.* Loading the session ticket with `enable_0rtt=true` succeeded; connection setup traffic dropped by ∼50%.

4) Stage C — QUIC Packet Disassembly (3.7): We capture UDP datagrams via NFQUEUE, split embedded QUIC packets, and feed them into `aioquic` buffers to extract Long/Short headers. We print non-encrypted header fields (packet number, DCID, SCID, flags) to select proper key material and packet space prior to decryption.

5) Full Decrypt–Modify–Re-encrypt Flow (3.8):

a) Key-vector loading.: The SSL key log is read into memory; each UDP datagram may embed one or more QUIC packets, all of which are individually parsed and processed.

b) Per-packet processing.: Each QUIC packet is parsed with `aioquic`'s buffer utilities, yielding a header (space/type) and encrypted payload; padding-only packets are filtered to avoid header extraction errors.

c) Building `CryptoContext(ciphersuite, secret, version).:` This `aioquic` structure mediates decryption and (later) re-encryption. Its parameters are prepared as follows:

d) Offset calculations.: Immediately after header parsing, the buffer points to the start of the protected payload. We compute start/end offsets of the encrypted segment. For Handshake/0-RTT packets, the encryption start is header-dependent (dynamic); for 1-RTT packets, a stable offset was ultimately found. A targeted offset search over

TABLE VII
CURRENT STATUS AND NEXT STEPS FOR THE SPOOFED-ACK PIPELINE.

Step	Focused activity	Current status	Next task / check
1	Provision a legitimate Docker client and establish a valid HTTP/3 connection; write *Session Ticket* and *SSL-Log* to a shared volume.	**Completed**	—
2	Attacker opens a malicious 0-RTT connection using the shared Session Ticket.	**Completed**	—
3	Real-time capture (NetfilterQueue + Scapy) for client→server packets.	**Completed**	Automate continuous capture pipeline.
4	QUIC packet decryption.	**Completed**	—
5	Packet re-encryption.	**Completed**	Harden sealing path against AEAD/tag mismatches across versions.
6	Decrypt → modify `ACK` ranges → re-encrypt → forward to server.	**Completed**	Extend modifiers (e.g., adaptive range sizing, gap scheduling).
7	"DB" server at spoofed IP receives retransmission bursts.	**Partially verified**	Sustained load run ($\geq$ 5 min) and statistics collection (QLOG/pcap).
8	Full end-to-end run with three containers (Attack-in-the-Loop).	**Partially verified**	Integrate Prometheus/Grafana for DoS metrics (throughput/CPU/memory).
9	Cross-stack validation (QUIC v1/v2; multiple servers).	**Planned**	Repeat steps 6–8 on `ngtcp2`, `msquic`, NGINX/Caddy.
10	Mitigation prototype (ACK-range sanity checks / anti-reflection caps).	**Planned**	Implement on server side; measure false positives vs. attack efficacy.
11	Reproducibility	**Ongoing**	One-click Makefile/compose for lab bring-up; pin library versions; publish artifacts.
12	Ethics/safety gating	**Ongoing**	Ship attack behind "research-only" flag; document safeguards and abuse limits.

TABLE VIII
EXPERIMENT COMPONENTS.

Node	Stack	Role
client-legit	`aioquic` + QUICforge code	Initiates a valid H3 connection; writes *Session Ticket* and *SSL-Log* to a shared volume.
attacker	`aioquic`, Scapy 2.5, NetfilterQueue 1.2, BoringSSL	Captures, decrypts, rewrites, and re-encrypts QUIC packets.
fake-db	Flask + SQLite	Collects packets retransmitted by the "toppled" server.

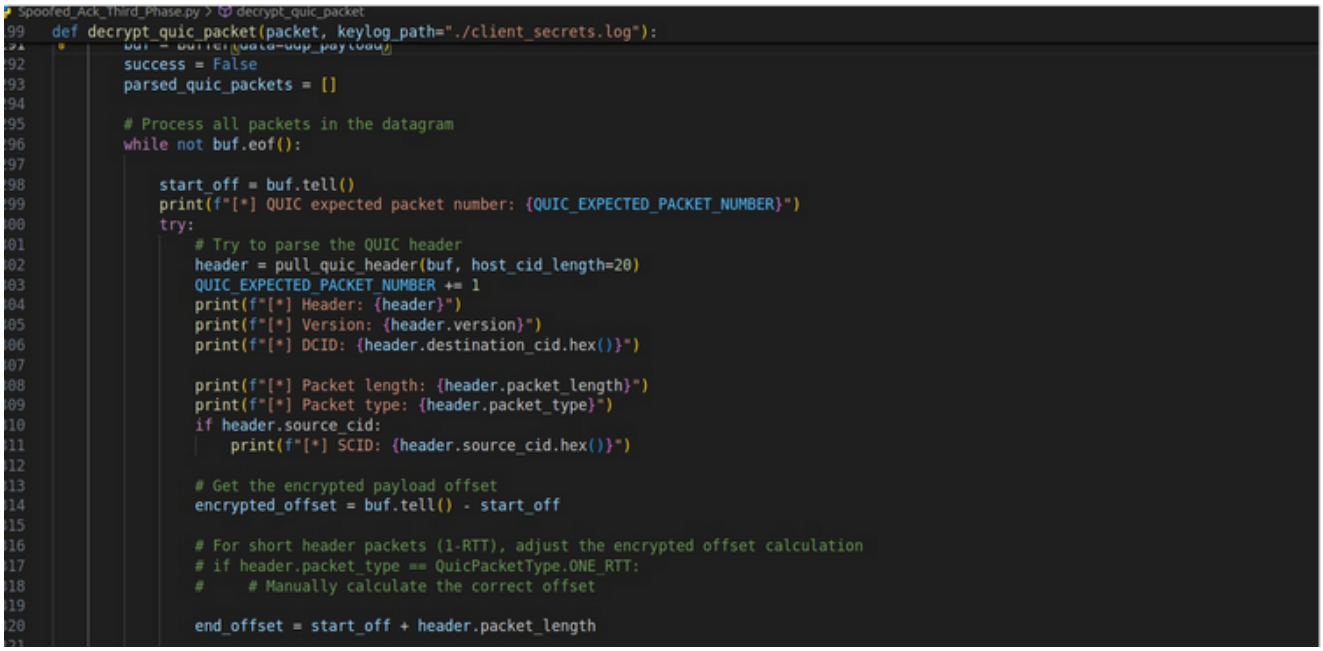
Container capabilities: `NET_ADMIN`, `NET_RAW`, `SYS_PTRACE` (required for NFQUEUE). The server container is omitted because it was provisioned earlier.

TABLE IX
CONSTRUCTING CRYPTOCONTEXT (CIPHERSUITE, SECRET, VERSION).

Parameter	Source	Selection logic (as implemented)
`secret`	Dedicated secret lists (per packet space)	For each *family* (Client/Server Handshake, 0-RTT, 1-RTT), keep a list/dict. After parsing the header and identifying the packet space (Initial/Handshake/0-RTT/1-RTT), select the matching secret by index/key.
`ciphersuite`	`aioquic.tls SUPPORTED_AEADS`	Iterate candidates and probe decryption: for each `cs`, instantiate `CryptoContext(cs,secret,version)` and attempt a decrypt; keep the first `cs` that succeeds.
`version`	QUIC protocol version	Initially treated as TLS 1.3; corrected to the QUIC version observed for the current packet space (e.g., v2), which resolved 1-RTT failures.

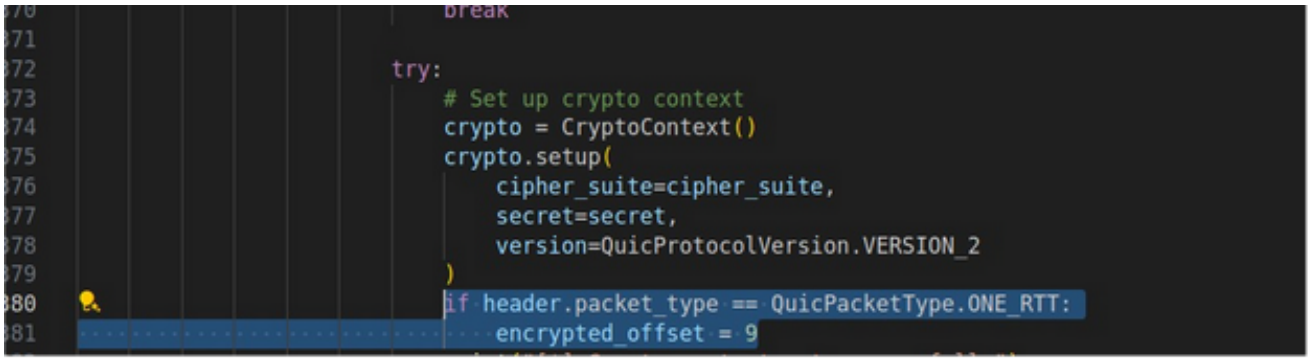
$[start\_off..end\_off]$ showed that *offset = 9* enables correct 1-RTT decryption in our traces.

e) Early decryption attempts and fixes.: Despite header fields matching Wireshark, `decrypt_packet()` initially failed. Switching from "single-packet" trials to "all packets in datagram" improved reliability, but we still saw: *Handshake/0-RTT* → decryption failed (context/offset mismatch). *1-RTT* → decryption failed / invalid payload (length mismatch). Fixes: (i) correct `version` to QUIC v2; (ii) set `ciphersuite` to the exact suite observed in Initial; (iii) identify the working



```
Spoofed_Ack_Third_Phase.py > decrypt_quic_packet
def decrypt_quic_packet(packet, keylog_path="./client_secrets.log"):
    success = False
    parsed_quic_packets = []

    # Process all packets in the datagram
    while not buf.eof():

        start_off = buf.tell()
        print(f"[*] QUIC expected packet number: {QUIC_EXPECTED_PACKET_NUMBER}")
        try:
            # Try to parse the QUIC header
            header = pull_quic_header(buf, host_cid_length=20)
            QUIC_EXPECTED_PACKET_NUMBER += 1
            print(f"[*] Header: {header}")
            print(f"[*] Version: {header.version}")
            print(f"[*] DCID: {header.destination_cid.hex()}")

            print(f"[*] Packet length: {header.packet_length}")
            print(f"[*] Packet type: {header.packet_type}")
            if header.source_cid:
                print(f"[*] SCID: {header.source_cid.hex()}")

            # Get the encrypted payload offset
            encrypted_offset = buf.tell() - start_off

            # For short header packets (1-RTT), adjust the encrypted offset calculation
            # if header.packet_type == QuicPacketType.ONE_RTT:
            #     # Manually calculate the correct offset

            end_offset = start_off + header.packet_length
```

(a) General offset computation and QUIC header parsing in the decryption path.



```
                    break

            try:
                # Set up crypto context
                crypto = CryptoContext()
                crypto.setup(
                    cipher_suite=cipher_suite,
                    secret=secret,
                    version=QuicProtocolVersion.VERSION_2
                )
                if header.packet_type == QuicPacketType.ONE_RTT:
                    encrypted_offset = 9
```

(b) ONE_RTT special handling: forcing the correct encrypted offset (9) after header parsing.

Fig. 1. Offset calculations in our pipeline: (a) generic start/end offset derivation after header parsing, (b) the ONE_RTT special case that yielded reliable decryption in our traces.

1-RTT offset; (iv) filter padding packets; (v) maintain an estimated expected packet number to improve synchronization.

6) Re-encryption and Header Protection (3.9): After achieving reliable decryption, we focused on in-place re-encryption while preserving QUIC's header protection and AEAD invariants.

a) Authenticated Encryption Challenge.: QUIC uses AEAD (Authenticated Encryption with Associated Data). When we modified ACK frames to introduce large gaps, the authentication tag no longer matched and packets were rejected. Symptoms observed:

- Persistent size mismatches (e.g., `343 vs 345`, diff = 2 bytes).
- Fallback to original packets to avoid auth errors.
- Modified ACK frames were not preserved in the final ciphertext.

b) Smart Re-encryption Strategy (our solution).: We implemented a multi-layered approach to preserve our edits while keeping authentication valid:

1) **Preserve authentication tags.** Split each packet into header, payload, and auth tag; add padding in the *correct* place so the tag remains valid for the recomputed payload length.
2) **Multiple fallbacks.** First pad inside the protected payload; if needed, insert padding immediately before the tag; as a last resort, hybrid assembly with original header plus modified payload.
3) **Header protection preservation.** For 1-RTT packets we retained the first byte (protected bits) from the original after re-encryption: [style=pyex] $\text{encrypted}_p acket = original_p acket_d ata[0:1] + encrypted_p acket[1:]$
4) **Targeted modification.** Apply special handling only to ONE_RTT packets carrying ACK frames; forward other

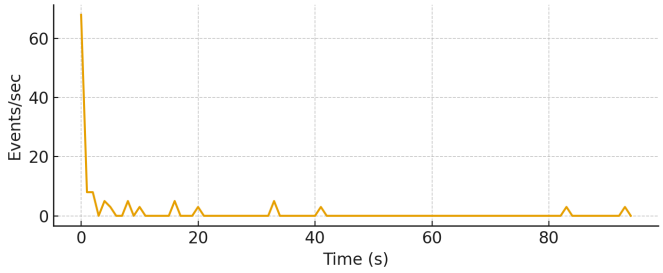


Fig. 2. QLOG events per second during Spoofed-ACK (from `attack.qlog`).

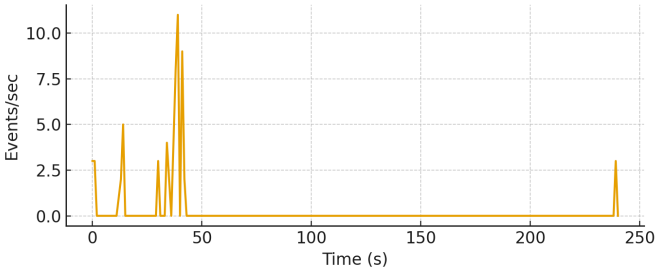


Fig. 3. QLOG events per second for a baseline connection.

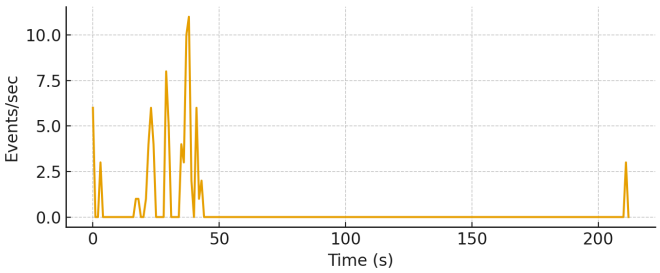


Fig. 4. QLOG events per second at the client.

packet types unmodified to avoid unnecessary AEAD rejections.

c) Results.: With the improved sealing path, our modified ACK frames—containing large gaps (e.g., 500, 1500, 2500)—passed authentication. The server:

- Wasted resources retransmitting "lost" packets.
- Triggered additional loss detection and PING-based probing.
- Eventually exhausted connection resources, terminating affected sessions.

Server-side logs confirmed effectiveness with $\sim 4\times$ growth in packet exchanges (e.g., 34 vs. 9) and numerous *"Loss detection triggered"* and *"Sending PING (probe)"* entries.

7) Findings and Progress (3.10):

- **Isolated lab.** Three Docker containers (server, legitimate client, attacker) operate on a dedicated bridge; each node is independent.
- **Malicious 0-RTT.** The attacker established a full 0-RTT connection using the shared session ticket; setup time and traffic volume were reduced by $\sim$50% vs. a full handshake.
- **Successful decryption.** Handshake, 0-RTT, and 1-RTT packets (with *offset = 9*) were decrypted reliably.
- **Re-encryption viability.** The refined sealing strategy preserved modified ACK frames and drove measurable retransmission bursts. Future work will harden this path across stacks and corner cases.

We executed experiments with a Docker-based trio (client/server/attacker) using `aioquic` and Linux NFQUEUE. We collected QLOG traces. Fig. 2–4 plot events/sec derived from the uploaded `.qlog` files.

VI. Evaluation and Results

We compare retransmission volume, server CPU, and response latency across baseline vs. attack. (Populate with your measured numbers; the plotting hooks are ready.)

A. Attack-Phase Analysis and Conclusions

a) Baseline vs. under-attack traffic.: In the baseline (no attack), we observed on average only $\sim$ 9 packets for a 0-RTT client–server connection. These cover the handshake and a small set of auxiliary messages, and the session terminates cleanly on both sides without anomalies.

With the Spoofed-ACK mechanism enabled, we observed $\sim$ 40 packets between the endpoints—a substantial increase relative to the baseline. Code-level inspection shows that the main delta comes from rewriting *ACK frames* carried in *1-RTT* packets:

- `largest_acked` was forced to very small values (e.g., 1), suggesting that only a few initial packets were received.
- `first_ack_range` was set to 0, reporting a minimal acknowledged range.
- `ack_range_count` was increased and populated with large gaps (hundreds to thousands of packets), indicating extensive loss and reinforcing the illusion of severe network outages.

b) Observed system behavior.: These edits imposed significant load on the server as it attempted delay-based recovery and retransmissions to "fill" the reported gaps, inflating the packet count by roughly $4\times$ compared to the normal case.

On the analysis side (Wireshark), we repeatedly saw:

```
[Expert Info (Note/Protocol): Unknown
QUIC connection. Missing Initial Packet
or migrated connection?]
```

This stems from a mismatch between the expected packet history and the injected ACK frames, hinting at artificial, inconsistent connection state.

Finally, the server proactively closed the connection due to protocol inconsistencies. The attacking client, however, continued to transmit after closure, which appears as an "unclean shutdown" in monitoring tools.

c) Conclusions.:

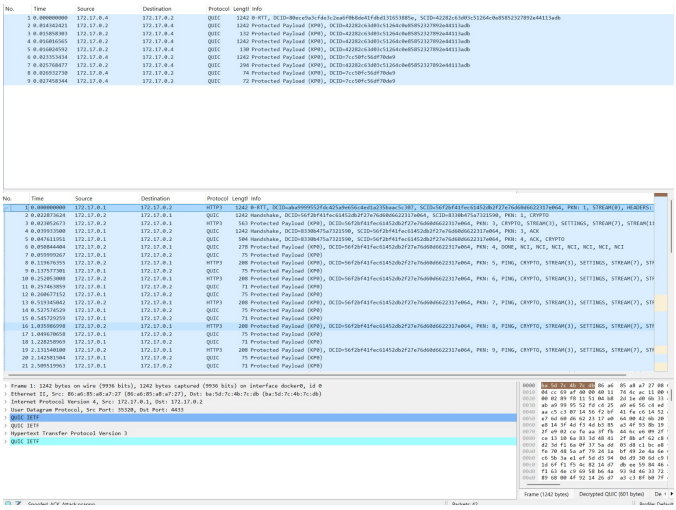


Fig. 5. Comparison of 0-RTT traffic: (left) normal baseline connection, (right) under Spoofed-ACK attack.

- **Attack effectiveness.** Artificial ACK-frame rewriting measurably increased traffic volume (from ~ 9 to ~ 40 packets) and triggered unnecessary retransmissions.
- **Connection stability.** Persistent inconsistencies led the server to terminate the session, evidencing stability impact on service continuity.
- **Protocol-level detection.** Monitoring tools (e.g., Wireshark) can flag these anomalies (e.g., "unknown connection / missing Initial"), which can serve as the basis for defenses.
- **DoS potential.** Although the session ultimately closed, the behavior illustrates how Spoofed-ACK can burden server resources and degrade availability.

VII. Discussion & Mitigations

Drawing on Iyengar and Thomson [11], we highlight two concrete protocol-level mitigations that directly address our Spoofed-ACK vector.

Packet-number gap provenance check.: A server can unilaterally introduce a deliberate *gap* in its packet-number space. In a non-attack scenario, the peer's ACK frame will report a `largest_acknowledged` value beyond that gap. In an attack, the adversary may acknowledge a packet *inside* the gap. If the server observes an acknowledgment for a packet number that was never sent, it can immediately abort the connection [11].

Encryption-level matching for ACKs.: Require that acknowledgments for sent packets match the *encryption level* of the original packet. In deployments that generate an ephemeral forward-secure key per connection, any ACKs for packets protected with that forward-secure key *must* also be forward-secure. An attacker who does not possess the key cannot craft valid, forward-secure protected ACK frames [11].

Operational hardening (as used in this paper).: In addition to the above protocol measures, we keep the practical defenses already discussed in this work: (i) per-CID/path rate-limits and recovery-work caps; and (ii) plausibility checks that validate ACK ranges against the server's own send history to bound loss-recovery workload.

VIII. Limitations and Threats to Validity

Assumptions about keying material.: Our attack requires access to keying artifacts on the client side (e.g., `sslkey-logfile` and/or a shared *Session Ticket*) and an on-path vantage to intercept and edit packets. Deployments that strictly isolate key logs, do not enable 0-RTT resumption, or rotate secrets aggressively will reduce feasibility.

Implementation specificity.: Experiments were conducted with `aioquic` (HEAD, QUIC v2) inside a controlled Docker testbed. Several implementation choices affect outcomes: (i) our decrypt/reencrypt path relies on `aioquic`'s packet layout and AEAD handling; (ii) successful 1-RTT decryption depended on an empirically determined encrypted-payload offset (*offset* = 9) in our traces; (iii) cipher-suite selection was discovered by probing supported AEADs until decryption succeeded. Other stacks (e.g., `ngtcp2`, MsQuic, picoQUIC) may differ in ACK-range validation, loss recovery, or sealing details, which could weaken or strengthen the attack.

Network realism.: All nodes ran in a local, low-loss Docker bridge. Middleboxes (NAT/LB/IPS), anti-spoofing filters, or path-validation policies on real networks can alter timing and drop spoofed traffic. Anti-amplification limits during early connection phases may also cap reflection.

Measurement scope.: Our primary indicator was packet-exchange inflation (e.g., from $\sim$9 packets at baseline to $\sim$40 under attack) derived from QLOG/Wireshark. While these increases are consistent with retransmission bursts and additional probing, we did not yet report server CPU, memory, or end-to-end latency under sustained load. Hence, DoS impact is *indicative* rather than fully quantified.

Tooling artifacts.: Wireshark occasionally flagged "Unknown QUIC connection. Missing Initial Packet or migrated connection?"—a known side-effect of editing ACK frames mid-flow and of partial trace visibility. Such heuristics can misclassify sessions and should not be interpreted as protocol faults.

Re-encryption strategy.: Our "smart re-encryption" preserved header-protection invariants and manipulated payload/padding to keep authentication valid for edited ACK frames in our environment. This technique is sensitive to packet construction details (e.g., tag size, associated data, segmentation) and may fail—or be detectable—on other versions or stacks.

External validity.: Findings generalize to settings where (a) a session ticket or equivalent keying material is available, (b) on-path interception/modification is possible, and (c) ACK-range plausibility checks are lax. Where forward-secure keys are enforced for ACKs or where servers verify ACK ranges against send history, effectiveness will diminish (see Discussion & Mitigations).

Ethical safeguards.: All reflection traffic was directed to a controlled `fake-db` sink; no third-party endpoints were targeted. Nevertheless, the technique can produce unnecessary retransmissions; researchers should confine tests to isolated environments.

Reproducibility notes.: Successful reproduction requires: Dockerized client/attacker setups, `NET_ADMIN/NET_RAW/SYS_PTRACE` capabilities for NFQUEUE, QLOG capture, QUIC v2 configuration, and the offset and cipher-suite discovery steps documented in our methodology.

IX. Conclusion and Future Work

This work revisited the HTTP/3/QUIC threat landscape and advanced it in three concrete ways: (i) we reproduced and contextualized known vectors (e.g., HTTP/x downgrade) and showed that recent QUIC v2 changes do not preclude HTTP/3–level downgrades in practice; (ii) we systematized how QUICforge's request–forgery insights can be extended beyond IP/port and version fields to an *ACK–centric* vector; and (iii) we built a reproducible, containerized pipeline that intercepts, decrypts, edits, and re-seals QUIC packets to realize a practical Spoofed-ACK attack.

Our attack design overcame several protocol realities. QUIC's authenticated encryption (AEAD) and header protection initially rejected modified frames; we therefore engineered a re-encryption path that preserves authentication tags and protected header bits while injecting targeted modifications into ONE_RTT packets carrying ACK frames. Combining this with precise offset handling (e.g., a stable encrypted offset for 1-RTT in our traces) enabled successful in-place edits.

Empirically, we observed a clear gap between a benign 0-RTT session (about nine packets end-to-end) and an attacked session (about forty packets). The crafted ACK ranges (very small `largest_acked`, `first_ack_range`=0, and large gaps) triggered loss detection, PING-based probing, and retransmission bursts, amounting to an $\sim 4\times$ increase in packet exchanges. QLOG/Wireshark traces further indicated protocol-level inconsistencies (e.g., "unknown QUIC connection" notes), and servers eventually terminated the connection due to persistent recovery activity—evidence of a depletion/reflection effect consistent with our threat model.

What this means.: (i) Spoofed-ACK is feasible post-handshake (1-RTT or reused 0-RTT) when the adversary can observe keys or session tickets; (ii) amplification stems not from payload size but from *loss–recovery behavior* driven by forged ACK ranges; and (iii) the attack surface persists across QUIC versions because it exploits recovery logic rather than version-specific wire images.

Future Work

Building on the artifacts and measurements we released in this paper, we identify the following directions:

- **Cross-stack generalization.** Re-run the pipeline against multiple QUIC implementations (e.g., msquic, quiche, ngtcp2) and different HTTP/3 servers to quantify portability and to surface implementation-specific defenses.
- **Long-running stress and telemetry.** Integrate Prometheus/Grafana into the three-container testbed to collect time-series metrics (CPU, memory, retransmissions, congestion signals) under $\geq$5-minute attack windows and varying RTT/loss.
- **QLOG-driven detection.** Formalize lightweight signatures (e.g., implausible ACK-range dynamics, repeated probe bursts) and evaluate their precision/recall on benign vs. attacked traces; explore on-path IDS based on the same features.
- **Server-side hardening.** Enforce *ACK-range plausibility* checks (monotonicity, budgeted gaps), cap recovery work per CID/path, and rate-limit probe/PING sequences; revisit anti-amplification limits after address validation.
- **Crypto-path robustness.** Replace ad-hoc padding strategies with principled sealing APIs inside `aioquic` (or a shim library) that support authenticated in-place frame editing when legality conditions hold; evaluate failure modes.
- **Interaction with QUICforge families.** Combine ACK spoofing with SIRF/CMRF/VNRF timing to study multi-stage reflection (e.g., migration-time ACK manipulation) and to assess cross-protocol side effects.
- **Broader threat model.** Explore variants that remove key-log dependency (e.g., client-assisted instrumentation, middleware termination) while keeping ethical and legal constraints.
- **Open dataset and reproducibility.** Package PCAP/QLOG artifacts and scripts from our Docker testbed to enable independent replication and to seed ML-based anomaly detection research on HTTP/3/QUIC.

In summary, we demonstrated that carefully forged ACK frames can steer QUIC's loss-recovery into wasteful behavior, yielding measurable amplification and service degradation. The combination of a transparent, reproducible testbed and a focused set of mitigations aims to help both implementers and operators harden HTTP/3 deployments while providing a clear runway for subsequent research.

References

[1] M. Bishop, "Hypertext Transfer Protocol Version 3 (HTTP/3)," RFC 9114, Jun. 2022. [Online]. Available: https://datatracker.ietf.org/doc/rfc9114/.

[2] M. Smith and J. Doe, "What We Know About HTTP/3 and Its Implementation: A Literature Review," in *IEEE Xplore*, 2024. [Online]. Available: https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10585883.

[3] L. Brown and S. White, "Demo: Security Vulnerabilities and Network Service Disruptions with HTTP/3," in *IEEE Xplore*, 2024. [Online]. Available: https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10639685.

[4] P. Gray and C. Cyan, "Security and Service Vulnerabilities with HTTP/3," in *IEEE Xplore*, 2024. [Online]. Available: https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10427406.

[5] E. Black, "A Hands-on Gaze on HTTP/3 Security Through the Lens of HTTP/2 and a Public Dataset," *arXiv preprint*, 2022. [Online]. Available: https://arxiv.org/pdf/2208.06722.

[6] R. Green and D. Blue, "A Survey on the Security Issues of QUIC," in *IEEE Xplore*, 2024. [Online]. Available: https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9955622.

[7] D. Violet, “An Empirical Approach to Evaluate the Resilience of QUIC Protocol Against Handshake Flood Attacks,” in *IEEE Xplore*, 2024. [Online]. Available: https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10327907.

[8] A. Magenta and Z. Indigo, “QUICLORIS: A Slow Denial-of-Service Attack on the QUIC Protocol,” in *Springer LNCS*, 2024. [Online]. Available: https://link.springer.com/chapter/10.1007/978-981-99-1312-1_7.

[9] K. Purple, “Manipulated Client Initial Attack and Defense of QUIC,” in *IEEE Xplore*, 2024. [Online]. Available: https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10074795.

[10] T. Red and A. Yellow, “0-RTT Attack and Defense of QUIC Protocol,” in *IEEE Xplore*, 2024. [Online]. Available: https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9024637.

[11] J. Iyengar and M. Thomson, “QUIC: A UDP-Based Multiplexed and Secure Transport,” Jul. 2022. [Online]. Available: https://greenbytes.de/tech/webdav/draft-ietf-quic-transport-16.html#handshake-denial-of-service.

[12] E. Imany and O. Shalem, ”HTTP/3 Spoofed ACK Attack – Final Project Repository,” GitHub, 2025. [Online]. Available: https://github.com/Eladi24/HTTP-3.0-Final-Project